

Question 1

Scenario: Pet Age Calculator

A veterinary clinic stores information about dogs. The veterinarian frequently needs to calculate the total age of two dogs to determine whether they belong to the same treatment category. Instead of manually adding their ages, the clinic wants to use the + operator.

Instructions

Create a class named **Dog** with the following private data members:

- name
- age

Your program should:

1. Implement a default constructor and a parameterized constructor.
2. Implement getter and setter methods.
3. Implement a showDetails() function to display a dog's information.
4. Overload the + **operator** using a **member function** so that adding two Dog objects returns the **sum of their ages**.
5. In the main() function:
 - Create two Dog objects.
 - Display their information.
 - Display the total age using the overloaded + operator.

Question 2

Scenario: Student Registration System

A university allows students to add their family name after completing registration. The software developer wants to use the + operator instead of writing a separate function.

Instructions

Create a class named **Student** with the following private data members:

- name
- studentID

Your program should:

1. Implement constructors, getters, setters, and a showDetails() function.
2. Overload the + **operator** using a **member function** that accepts a string.
3. The operator should append the given string to the student's existing name and return the updated name.
4. In the main() function:
 - Create one student object.
 - Display the student's information before and after adding the family name using the overloaded operator.

Question 3 (Intermediate)

Scenario: Company Employee Management

A company stores information about its employees. The HR department wants to simplify different salary calculations using operator overloading.

Instructions

Create a class named **Employee** with the following private data members:

- name
- age
- salary

Your program should:

1. Implement constructors, getters, setters, and a showDetails() function.
2. Overload the + **operator** using **member functions** for the following operations:
 - **Employee + Employee**
 - Return a **new Employee object** whose:
 - Name is the combination of both employee names.
 - Age is the sum of both ages.
 - Salary is the sum of both salaries.
 - **Employee + double**

- Return the employee's salary after adding the bonus amount.
3. In the `main()` function:
- Create two employee objects.
 - Demonstrate both overloaded operators.
 - Display the resulting employee object and updated salary.

Question 4 (Hint friend function)

Scenario: Smart Bank Customer Management

A bank stores information about its customers. The bank wants its software to support flexible arithmetic operations using **friend function operator overloading**, allowing both the object and a built-in type to appear on either side of the `+` operator.

Instructions

Create a class named **Customer** with the following private data members:

- name
- age
- balance

Your program should implement the following **friend functions**:

1. **Customer + double**
 - Return the customer's updated balance after adding the specified amount.
2. **double + Customer**
 - Return the same updated balance.
 - *(This operation must work successfully.)*
3. **Customer + Customer**
 - Return a **new Customer object** whose:
 - Name is the combination of both customer names.
 - Age is the total age.
 - Balance is the combined balance.
4. Overload **prefix ++**

- Increase age by **1 year**.
 - Increase balance by **5000**.
 - Return the updated object.
5. Overload **postfix ++**
- Increase age and balance in the original object.
 - Return the **old copy** of the customer.
6. In the `main()` function, demonstrate every overloaded operator and display the results after each operation.

Question 5 (Hint- Native/Member Function)

Scenario: University Faculty Management System

A university is developing a faculty management system. Each faculty member has personal and salary information. The university wants to implement operator overloading **using member (native) functions only**.

Instructions

Create a class named **Faculty** with the following private data members:

- name
- age
- salary

Your program should implement the following **member function operator overloads**:

1. **Faculty + double**
 - Return the updated salary after adding the bonus amount.
2. **Faculty + Faculty**
 - Return a **new Faculty object** whose:
 - Name is the combination of both faculty names.
 - Age is the sum of both ages.
 - Salary is the combined salary.
3. Overload **prefix ++**

- Increase age by **1 year**.
 - Increase salary by **10000**.
 - Return the updated object.
4. Overload **postfix ++**
- Return the faculty member's previous information before updating.
 - Then increase age by **1 year** and salary by **10000**.
5. In the `main()` function:
- Demonstrate all overloaded operators.
 - Attempt to write the following statement:

`5000 + faculty;`

6. Explain **why this statement generates a compile-time error** when operator overloading is implemented using **member (native) functions**, even though the following statement works correctly:

`faculty + 5000;`

Note: Do **not** use friend functions in this question. Implement every overloaded operator as a **member function**, similar to the `operatorOverLoadingWithNativeFunction.cpp` example.